Question and ans

C# (10 Questions)

1. **What is C#?**
   A modern, object-oriented language by Microsoft. Used for building web, desktop, and cloud apps.
   Benefit: Strong typing, rich libraries, cross-platform support.
2. **Why use C# in enterprise apps?**
   It integrates seamlessly with .NET and Azure, making it ideal for scalable business solutions.
3. **Explain value types vs reference types in C#.**
   Value types store data directly (int, struct). Reference types store pointers (class, object).
   Benefit: Helps optimize memory usage.
4. **What is dependency injection in C#?**
   A design pattern for loose coupling.
   Example: Injecting a repository into a controller.
5. **Explain async/await in C#.**
   Used for asynchronous programming.
   Benefit: Non-blocking I/O, better performance.
6. **What are generics in C#?**
   Allow type-safe reusable code.
   Example: `List<T>`.
7. **Difference between interface and abstract class?**
   Interface defines contract, abstract class provides partial implementation.
8. **What is LINQ in C#?**
   Language Integrated Query for querying collections.
   Example: `var result = users.Where(u => u.Age > 18);`
9. **Explain exception handling in C#.**
   Using try-catch-finally blocks.
   Benefit: Graceful error recovery.
10. **When would you use structs instead of classes?**
    For lightweight objects that don’t require inheritance.

MVC (10 Questions)

11. **What is MVC?**
    Model-View-Controller pattern for separation of concerns.
    Benefit: Easier maintainability.
12. **Why use MVC in ASP.NET?**
    Provides structured development with clear separation between UI and logic.
13. **Difference between MVC and MVVM?**
    MVVM uses ViewModel for binding (common in WPF). MVC is simpler for web.

14. **What is a Controller in MVC?**
    Handles user requests and returns responses.
    Example: `ProductController` managing product actions.
15. **What is Razor in MVC?**
    A view engine for embedding C# in HTML.
    Example: `@Model.Name`.
16. **Explain routing in MVC.**
    Maps URLs to controller actions.
    Example: `/products/details/1`.
17. **What is TempData in MVC?**
    Stores data temporarily between requests.
    Benefit: Useful for redirects.
18. **Difference between ViewBag and ViewData?**
    Both pass data to views. ViewBag is dynamic, ViewData is dictionary-based.
19. **What is Partial View in MVC?**
    Reusable view component.
    Example: Header/footer sections.
20. **When is MVC not ideal?**
    For small apps where overhead is unnecessary.

Web API (10 Questions)

21. **What is Web API?**
    Framework for building RESTful services.
    Benefit: Lightweight, scalable.
22. **Difference between Web API and WCF?**
    Web API is REST-based, WCF is SOAP-based.
23. **Why use REST APIs?**
    Standardized, stateless, scalable.
    Example: Payment gateway integration.
24. **What is HTTP Verb in Web API?**
    Defines action: GET, POST, PUT, DELETE.
25. **What is content negotiation in Web API?**
    Determines response format (JSON/XML).
26. **Explain versioning in Web API.**
    Allows multiple API versions.
    Example: `/api/v1/products`.
27. **What is token-based authentication in Web API?**
    Uses JWT tokens for secure access.
28. **Difference between Web API and MVC Controller?**
    Web API returns data (JSON/XML), MVC returns views.
29. **What is Swagger in Web API?**
    Tool for documenting APIs.
    Benefit: Easy testing.
30. **When to use Web API vs gRPC?**
    Web API for REST, gRPC for high-performance communication.

SQL Server (10 Questions)

31. **What is SQL Server?**
    Microsoft's relational database system.
    Benefit: Reliable, secure, scalable.
32. **Difference between clustered and non-clustered index?**
    Clustered defines physical order, non-clustered is logical.
33. **What is normalization?**
    Organizing data to reduce redundancy.
    Benefit: Efficient queries.
34. **What is denormalization?**
    Adding redundancy for performance.
35. **What are stored procedures?**
    Precompiled SQL code.
    Benefit: Reusability, security.
36. **What is a trigger in SQL Server?**
    Executes automatically on events (INSERT/UPDATE/DELETE).
37. **What is transaction in SQL Server?**
    Ensures atomicity of operations.
    Example: Bank transfer.
38. **Difference between DELETE and TRUNCATE?**
    DELETE removes rows with logging, TRUNCATE removes all rows quickly.
39. **What is SQL Server security model?**
    Role-based access, encryption.
40. **When to use SQL Server over NoSQL?**
    For structured relational data.

EF Core (10 Questions)

41. **What is EF Core?**
    ORM for .NET.
    Benefit: Simplifies database access.
42. **Difference between Database First and Code First?**
    Database First: Generate models from DB.
    Code First: Generate DB from models.
43. **What is DbContext in EF Core?**
    Manages database connections and queries.
44. **What are migrations in EF Core?**
    Track schema changes.
    Example: Adding new tables.
45. **What is lazy loading in EF Core?**
    Loads related data only when accessed.
46. **Difference between eager and lazy loading?**
    Eager loads immediately, lazy loads on demand.

47. **What is LINQ-to-Entities?**
    Querying database using LINQ.
    Example: `db.Products.Where(p => p.Price > 100).`
48. **What is shadow property in EF Core?**
    Property not defined in class but tracked in DB.
49. **What is EF Core performance optimization?**
    Use compiled queries, caching.
50. **When not to use EF Core?**
    For extremely performance-critical raw SQL operations.

888888888888888888888888888888888888888888888888

C# (Core Concepts)

1. What is C#?

C# is an object-oriented programming language developed by Microsoft for building applications on .NET.

2. What is CLR?

Common Language Runtime manages memory, execution, and garbage collection.

3. What is OOP?

Concept using classes and objects (Encapsulation, Inheritance, Polymorphism, Abstraction).

4. Difference between `ref` and `out`?

- `ref` → must initialize before passing
- `out` → no need to initialize

5. What is async/await?

Used for asynchronous programming to improve performance.

ASP.NET MVC

6. What is MVC?

Model-View-Controller pattern separates concerns.

7. What is Model?

Represents data and business logic.

8. What is View?

UI layer (HTML + Razor).

9. What is Controller?

Handles request and returns response.

10. What is Razor?

View engine used in MVC (`.cshtml`).

11. What is ViewBag vs ViewData?

- ViewBag → dynamic
- ViewData → dictionary

12. What is TempData?

Used for passing data between requests.

13. What is Routing?

Maps URL to controller action.

14. Attribute routing?

Defines routes using attributes.

15. Filters in MVC?

Authorization, Action, Result, Exception filters.

ASP.NET Web API

16. What is Web API?

Framework to build RESTful services.

17. Difference between MVC and Web API?

- MVC → UI + Logic
- Web API → only services (JSON/XML)

18. What are HTTP verbs?

GET, POST, PUT, DELETE.

19. What is REST?

Architectural style for APIs.

20. What is IActionResult?

Standard response type in Web API.

21. What is Model Binding?

Maps request data to parameters.

22. What is DTO?

Data Transfer Object for secure data transfer.

23. What is Swagger?

API documentation tool.

24. What is Dependency Injection?

Inject dependencies instead of creating objects manually.

25. What is Middleware?

Handles request pipeline in ASP.NET Core.

Entity Framework Core

26. What is EF Core?

ORM by Microsoft to interact with database using C#.

27. Code First vs Database First?

- Code First → DB from code
- DB First → code from DB

28. What is DbContext?

Main class to interact with database.

29. What is DbSet?

Represents table.

30. What are migrations?

Track and apply database changes.

31. What is Lazy Loading?

Loads data when needed.

32. What is Eager Loading?

Loads related data using `Include()`.

33. What is Explicit Loading?

Manually load related data.

34. What is LINQ?

Query language for data manipulation.

35. What is tracking vs no-tracking?

- Tracking → EF tracks changes
- No-tracking → read-only

SQL Server

36. What is SQL Server?

Relational database by Microsoft.

37. What is normalization?

Process to reduce redundancy.

38. Types of joins?

Inner, Left, Right, Full.

39. What is index?

Improves query performance.

40. What is stored procedure?

Precompiled SQL code.

41. What is primary key?

Unique identifier.

42. What is foreign key?

Maintains relationship between tables.

Azure (Cloud + Deployment)

43. What is Azure?

Cloud platform by Microsoft.

44. What is App Service?

Platform to host web apps.

45. What is Azure SQL Database?

Cloud-based SQL Server.

46. What is Deployment?

Publishing app to server/cloud.

47. Steps to deploy MVC/Web API to Azure?

1. Create App Service
2. Publish from Visual Studio
3. Configure connection string
4. Deploy

48. What is CI/CD?

Automated build and deployment pipeline.

49. What is Azure DevOps?

Tool for CI/CD and project management.

50. What is connection string in Azure?

Used to connect app with Azure SQL DB.

BONUS (Important Real-World Questions)

How MVC connects to Web API?

MVC calls Web API using HttpClient.

How EF Core used in Web API?

Inject DbContext → perform CRUD operations.

How to secure API?

Use JWT Authentication.

How to deploy full project?

- Backend → Azure App Service
- Database → Azure SQL
- Frontend → same App Service